

Bileşen-tabanlı Yazılım Mühendisliği

Yazılım Mühendisliği Dersi

Konu başlıkları

- Yeniden Kullanılabilir Yazılım
- Bileşen Kavramı
- Günümüz Bileşen Teknolojileri
- Bileşenler ve bileşen modelleri
- BTYM süreci

Yazılım yeniden kullanılabilirliği

- Buradaki temel fikir, daha önceden geliştirilmiş yazılım parçalarını kullanarak sistemi inşa etmektir.
- Yazılım yeniden kullanılabilirliği farklı seviyelerde olabilir.

Yazılım yeniden kullanılabilirliği

- Kaynak Kod Kopyalamak
 - En düşük seviyede yeniden kullanmak: kaynak kod kopyalamaktır.
 - Bu kullanımda, kopyalanmış kısımlar yazılımın içinde bir virus gibi yayılırlar.
 - Eğer kopyalanmış kod içinde bir gereksinim değişirse yada bir bug çıkarsa, tüm istemciler değişiklikten etkilenir ve güncellenmeleri gerekir.

Yazılım yeniden kullanılabilirliği

- Fonksiyon Kütüphaneleri
 - Kod fonksiyonun içindedir ve içerdeki herhangi bir değişiklik istemcileri etkilemez.
 - Ancak, fonksiyon kütüphaneleri genişletilebilir değildir.
 - Giriş veya çıkışlar üzerinde yapılabilecek herhangi bir değişiklikten istemciler etkilenebilir.

Yazılım yeniden kullanılabilirliği

- Sınıf Kütüphaneleri
 - Sınıf kütüphaneleri daha yüksek seviyede bir yeniden kullanılabilirlik sunar.
 - Genişletilebilirdirler ve nesne yönelimli yaklaşım davranışını gösterirler.
 - Ancak, sınıflar kullanılmadan önce çok iyi kavranması gerekir.

Yazılım yeniden kullanılabilirliği

- Yeniden kullanılabilirlik farklı diller arasında geçiş yapmayı desteklemiyor.
- Dolayısıyla farklı platformlar arası geçiş te mümkün görünmüyor.
- Bileşenler yüksek seviyede yazılım yeniden kullanılabilirliği sunuyorlar.
- İç kısımda bileşenlerin uygulaması dış dünyaya kapalı.
- Bilgi saklamanın en yüksek seviyesi. İstemciler sadece arayüzlere güveniyorlar.
- Arayüzler değişmediği sürece, bileşenler iç kısımda, istemcileri etkilemeden değiştirilebilirler.
- Bu nedenle, sistem geliştirmek için bileşen kullanmak, yeniden kullanılabilirliğin tüm gücünden yararlanmak için en ümit verici yaklaşım.

Bileşen-tabanlı Geliştirme

- Bileşen-tabanlı yazılım mühendisliği yazılım yeniden kullanılabilirliğine dayanır.
- Nesne-yönelimli geliştirmenin yeniden kullanma konusunda etkin olamayışından ortaya çıkmıştır.
- Bileşenler nesne sınıflarından daha soyuttur ve bağımsız hizmet sağlayıcıları olarak düşünülebilir.

BTYM Gereksinimleri

- Bağımsız bileşenler
- Bileşen standartları
- Ara yazılım
- Geliştirim süreci

BTYM tasarım ve geliştirme

- Bileşenler bağımsızdırlar, birbirlerine etkileri yoktur.
- Bileşen gerçekleştirimi açık değildir.
- İletişim iyi tanımlanmış arayüzlerle sağlanır.
- Bileşen platformları paylaşılmıştır ve geliştirim maliyetini azaltırlar.

BTYM problemleri

- *Bileşen güvenilirliği* - Kaynak kodu açık olmayan bir bileşene ne kadar güvenilebilir?
- *Bileşen sertifikasyonu* - Bileşenlerin kalitesini kim belirleyecek?
- *Beklenmeyen özellik tahmini* - Bileşen birleştirmede ortaya çıkan beklenmeyen özellikler nasıl tahmin edilebilir?

Bileşenler

- Bileşenler nerede çalıştığı ve geliştirildiği programlama dilinden bağımsız olarak servis sağlar
 - Bir bileşen bir yada daha fazla çalıştırılabilir nesneden oluşmuş, bağımsız çalıştırılabilir bir varlıktır.
 - Bileşen arayüzü yayınlanmıştır ve tüm etkileşimler yayınlanmış arayüz ile yapılır.

BİLEŞEN KAVRAMI

- Bileşenler,
 - değişik zamanlarda
 - farklı platformlarda (işletim sistemleri & donanımlar)
 - farklı programlama dilleri ile,
 - birbirinden bağımsız
- tasarlanmış yazılım parçalarıdır.

BİLEŞEN KAVRAMI

- İç kısımdaki uygulama kullanıcıya kapalıdır.
- İşlevselliklerini arayüzleri üzerinden sunarlar.
- Bileşen yaklaşımının en önemli noktası, yazılım bileşenlerinin etkin biçimde çalışabilmesi için bileşen arayüzlerinin standardize etmektir.

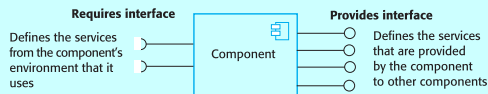
Bir hizmet sağlayıcı olarak bileşen

- Bileşen bağımsız çalıştırılabilir bir varlıktır. Diğer bileşenlerle kullanılmadan önce derlenmesine gerek yoktur.
- Bileşen tarafından sunulan servisler arayüz aracılığıyla kullanılır ve tüm bileşen etkileşimleri bu arayüzlerle gerçekleştirilir.

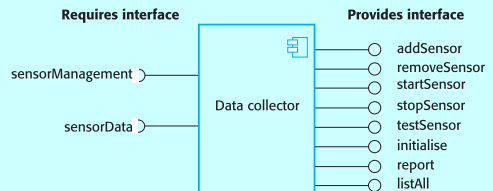
Bileşen arayüzleri

- Arayüz sağlar
 - Bileşen tarafından diğer bileşenler için sağlanan hizmetleri tanımlar/belirtir.
- Arayüze gereksinim duyar
 - Bileşenin belirtildiği şekilde çalışması için hangi hizmetlerin bileşen için hazır bulunması gerektiğini belirtir.

Bileşen arayüzleri



Veri toplayıcı bileşeni



Bileşenler ve nesneler

- Bileşenler dağıtılabılır varlıklardır.
- Bileşen gerçekleştirimi şeffaf değildir.
- Bileşenler dil bağımsızdır.
- Bileşenler standartlaştırılmıştır.

Bileşen modelleri

- Bir bileşen modeli bileşenin gerçekleştirimi, belgelenmesi ve dağıtımı için standartların tanımıdır.
- Bileşen modeli örnekleri
 - EJB model (Enterprise Java Beans)
 - COM+ model (.NET model)
 - Corba Component Model
- Bileşen modeli arayüzlerin nasıl tanımlanacağı ve arayüz tanımında hangi bölümlerin bulunacağını belirler.

CORBA (Genel Nesne İstek Aracı Mimarisi)

- OMG (Object Management Group - Nesne Yönetim Grubu) tarafından 1993 yılında geliştirilmiş.
- Bileşenler arasındaki arayüzleri tanımlamak için standart bir mekanizmaya sahiptir,
- CORBA, bir uygulamanın veya farklı uygulamaların çeşitli bileşenlerinin, kendi aralarında haberleşebilmeleri için gerekli olan tüm altyapıyı sağlar.

CORBA (Genel Nesne İstek Aracı Mimarisi)

- CORBA, bilgisayar yazılım dünyasında nadir olarak sağlanabilen "platform bağımsızlığı" ve "dil bağımsızlığı" özelliklerini içerir.
- Platform bağımsızlığı, üzerinde CORBA ORB yürütmesi olan her türlü platform (modern işletim sistemleri) üzerinde CORBA nesnelerinin kullanılabilmesi demektir.
- Dil bağımsızlığı ise, CORBA nesnelerinin hemen hemen istenilen tüm programlama dillerinde gerçekleştirilebileceği, haberleştikleri diğer CORBA nesnelerinin hangi dili kullandıklarını bilmek zorunda olmadıkları anlamına gelir.

ORB (Nesne İstek Aracı)

- Bir CORBA sisteminin en önemli parçası Nesne İstek Aracı (ORB)'dir.
- ORB, bileşenler arasında istemci-sunucu ilişkilerini sağlamak için gerekli tüm mekanizmalardan sorumludur.
- ORB,
 - istemci arayüzünün görünüşü,
 - nesnenin nerde konumlandığından,
 - hangi programlama dilinde yürütüldüğünden,
 - veya nesnenin arayüzüne yansımayan herhangi başka bir durumdantamamen bağımsızdır.

DCOM (Dağıtık Bileşen Nesne Modeli)

- 1993 yılında COM (Bileşen Nesne Modeli) Microsoft tarafından duyurulmuştur.
- Amacı, farklı sistemler üzerindeki bileşenlere erişim sağlamak.
- DCOM, bileşenlerden oluşan ağ uygulamaları yaratmayı mümkün hale getirerek 1996 yılında dağıtık sistemler piyasasına giriş yapmıştır.
- DCOM, CORBA ile benzer imkanlara sahiptir.
- DCOM Microsoft işletim sistemlerine (Windows 9x, NT, 2000) entegre edilmiştir. Windows işletim sistemleri dışında seyrek kullanılır.

DCOM (Dağıtık Bileşen Nesne Modeli)

- DCOM, COM (Component Object Model)'dan geliştirilerek oluşturulmuştur.
- COM, yazılım bileşenlerini geliştirmek ve yaymak amacıyla kullanılan nesne tabanlı bir çatıdır.
- COM, geliştiricilerin soyutlamaları bileşen arayüzleri olarak elde etmelerine izin verir, ve bu arayüzleri yürüten sınıfları sağlar.
- İstemci uygulamaları sadece bir nesnenin arayüzünde tanımlı olan fonksiyonları çağırabilirler.

Bileşen Tabanlı Yazılım Geliştirme Süreci



Bileşen belirleme sorunları

- **Güven** Bileşenin sağlayıcısına güvenmeye gereksinim duyarsınız. En iyi ihtimalle güvenilmeyen bir bileşen tanıtıldığı işi yapmaz. En kötü ihtimal ise güvenliğinizin kınılmasıdır.
- **Gereksinimler** Farklı bileşen grupları farklı ihtiyaçları karşılar.
- **Doğrulama**
 - Bileşen belirtimi detaylı testler geliştirmek için yeterli kadar detaylandırılmamıştır.
 - Bileşenler istenmeyen fonksiyonallığa sahip olabilirler.

Ariane launcher failure

- In 1996, the 1st test flight of the Ariane 5 rocket ended in disaster when the launcher went out of control 37 seconds after take off.
- The problem was due to a reused component from a previous version of the launcher (the Inertial Navigation System) that failed because assumptions made when that component was developed did not hold for Ariane 5.
- The functionality that failed in this component was not required in Ariane 5.

Bileşen Tabanlı Yazılım Geliştirme

- **Avantajlar**
 - Maliyet azalıyor
 - Risk düşüyor
 - Yazılımlar daha hızlı gerçekleştiriliyor
- **Dezavantajlar**
 - Gereksinimlerin uyarlanması gerektiğinden kullanıcının gerçek gereksinimlerinin karşılanmasından uzaklaşabiliyor.
 - Yazılımların geliştirilmesinde hazır kullanılan yazılım bileşenlerinin sahipliği dışında olduğundan bakım güçlükleri çekilebiliyor.

Yazılım Modelleme

- Karmaşık yapıyı daha kolay, anlaşılabilir parçalara bölmek ve canlandırmak için modellemek ve daha sonra bu parçaları aralarındaki ilişkileri inceleyerek bir bütün haline getirmek.
- Bir model oluşturmak geliştiriciye ya da programcıya proje çalışması içinde detay bilgisine boğulmadan temel konuyla ilgili zihninde bir resim oluşmasına olanak verir.

Yazılım Modelleme

- Modeller ;
 - Problemin kavranmasında,
 - Proje ile ilgili herhangi bir iletişimde (müşteri, analizci, tasarımcı, programcı...),
 - Döküman hazırlanmasında,
 - Tasarım ve problem çözümünde,küçümsenmeyecek kolaylıklar sağlar.
- Modeller, karmaşık sistem ya da yapıların gereksiz ayrıntılarından, kurtarılmasını sağlayıp daha kolay anlaşılmasını sağlarlar.
- Buna kısaca literatürde "Soyutlama (Abstraction) " denilmektedir.

UML

- Yazılım geliştirme ve üretim alanında ortak bir notasyon gereksinimi özellikle 90'lı yıllarda iyiden iyiye bir ihtiyaç haline gelmiştir.
- OMT, Boach ve DOSE
- UML(Unified Modelling Language)
- İlk resmi sürümü 1995 Ekiminde (UML version 08) ilan edilmiştir.
- 1997 yılında OMG (Object Management Group) UML'i sahiplendi ve açık standart olarak geliştirmeye başladı.

UML

- Yazılım Mühendisliğinde nesne tabanlı sistemleri modellemede kullanılan **açık standart** Görsel Modelleme dilidir.
- UML standartlaşmış notasyonlar ve görsel öğeler sunar.
- UML analiz ve tasarımın birimlerini standardize etme girişim olarak da kabul edilebilir.
- UML şu anda RATIONAL, MS VISIO, TOGETHER SOFT.. Vb bir çok modelleme aracı tarafından sunulmaktadır.

UML

- **UML** temelde aşağıdaki fonksiyonları yerine getirir.
 - Görsel Tasarım
 - Tanımlama
 - Yapılandırma
 - Dökümantasyon
- Geliştirme hayat döngüsü boyunca olan tüm işlemlerle ve farklı uygulama teknolojileri ile kullanılabilir.

Ödev

- UML Diyagramları:
 - Use Case Diagrams
 - Class Diagrams
 - Sequence Diagrams
 - Collaboration Diagrams
 - Object Diagrams
 - Activity Diagrams
 - Statechart Diagrams
 - Component Diagrams
 - Deployment Diagrams

Kaynak: Ian Sommerville, Software Engineering 8th Edition